Critical Reproduction of a Machine-Learning Intrusion Detection System

Do the Author's 99.87% Accuracy Claims Survive a Held-Out Test Set?

A Reproduction Study on the NSL-KDD Dataset

Course: Data Science in Cybersecurity

Instructor: Dr. Uri Itai

Student: Renata Haiek

Student ID: 212744916

Repository: https://github.com/RenataH02/IDS_DataScienceCyber.git

Table of Contents

Executive Summary

This project reproduces and critically evaluates the Medium article *"Enhancing Cybersecurity with Machine Learning: Analyzing Intrusion Detection Systems Using the NSL-KDD Dataset"* (R. Srivastava, March 2025). The article treats intrusion detection as a binary classification problem, normal versus attack, on the NSL-KDD dataset. It reports that a Random Forest reaches 99.87% accuracy and is the best model, that SVM and Decision Trees "also performed well," and that Naïve Bayes had the lowest accuracy. No code, train/test split, or preprocessing details are given, and although the article names Precision, Recall and F1, it never reports a value for any of them.

I rebuilt the full pipeline from scratch and evaluated it once on the official held-out test set (KDDTest+). None of the article's central claims held up.

- The 99.87% figure does not reproduce. My Random Forest reaches 76.71% accuracy on the held-out test set, a gap of 23.16 percentage points. That headline figure instead matches my 5-fold cross-validated training F1 of 0.9988, which strongly suggests the author measured training performance rather than test performance.
- "SVM performed well" is wrong. Linear SVM was the weakest of the six models on every metric (accuracy 74.97%, F1 0.7351, MCC 0.557).
- "Naïve Bayes had the lowest accuracy" is also wrong. Naïve Bayes (77.16% accuracy, F1 0.7692) beat both Logistic Regression and SVM.
- The dangerous failures are hidden by the article's metric choice. Random Forest misses 4,975 attacks, which is 22.07% of all attacks, and most of those misses fall in the R2L and U2R categories, the rarest but most serious intrusions. Recall, false negatives, and class imbalance are never discussed.

My extended pipeline adds the methodology the article leaves out: Spearman-based EDA justified by the heavy skew in the data, label encoding, redundancy removal at $|\text{Spearman } r| > 0.95$, model-appropriate scaling, six classifiers, seven metrics, stratified cross-validation, and a full error analysis. The strongest model was XGBoost, at 81.00% accuracy and F1 0.8048. That is a believable result for NSL-KDD and nowhere near the article's claim. My overall conclusion is that the original work is not reproducible and its headline claims are not supported by the evidence, so it should not be used as a methodological reference for IDS design.

1. Summary of the Source

1.1 The problem and why it matters

An Intrusion Detection System inspects network traffic and flags connections that are likely to be malicious. The article addresses the binary version of this task: given the features of one network connection, decide whether it is normal or an attack. The problem matters because networks produce far more traffic than analysts can review by hand, and an intrusion that slips through can lead to data

theft, privilege escalation, and a long-term foothold inside the network. Reliable automated detection, with a controlled rate of false alarms, is therefore a core need in any security operations team.

1.2 The proposed solution

The article proposes a standard supervised-learning pipeline. It loads NSL-KDD, trains several off-the-shelf classifiers (Logistic Regression, Decision Tree, Random Forest, SVM and Naïve Bayes), and compares them by accuracy. The author concludes that Random Forest is the best model and reports a single headline figure of 99.87% accuracy.

1.3 The dataset

NSL-KDD is the cleaned successor to the KDD Cup 1999 dataset, distributed by the Canadian Institute for Cybersecurity. It removes the duplicate and redundant records that inflated results on the original. The two files used here are `KDDTrain+` (125,973 rows) and `KDDTest+` (22,544 rows). Each has 43 columns: 41 features, a textual `label`, and a `difficulty_level` curation field. The 41 features fall into three semantic groups: basic TCP/IP attributes, payload-derived content features, and traffic statistics computed over a two-second window. Attacks belong to four families: DoS, Probe, R2L and U2R.

1.4 The methodology employed by the author

As far as the prose allows me to reconstruct it, the method is to read the data, apply some unspecified preprocessing, train five classifiers with default settings, and report accuracy. The article does not state an encoding strategy, a scaling choice, any feature selection, the train/test split, the validation scheme, or any hyper-parameters, and no source code or repository is provided.

2. Critical Evaluation of the Author's Claims

This section is the core of the project. I tested each headline claim by rebuilding the pipeline independently and evaluating on the official held-out test set. The table below records the verdicts, and the subsections that follow justify them.

Claim made by the article	What I observed	Verdict
Random Forest reaches 99.87% accuracy (best model)	RF accuracy = 76.71% on KDDTest+ (gap 23.16 pp)	Fails to reproduce
"SVM and Decision Trees also performed well"	SVM is the weakest model on every metric	Wrong
"Naïve Bayes had the lowest accuracy"	NB beats both LR and SVM	Wrong
Uses Precision, Recall, F1	Named, but no values given	Unverifiable
MCC / ROC-AUC	Absent; added in this study	Important gap
Class imbalance handled	Not mentioned	Gap
Cross-validation	Absent	Gap

Claim made by the article	What I observed	Verdict
FP / FN trade-off	Absent	Serious gap

2.1 The 99.87% accuracy claim looks like training accuracy

My Random Forest, trained with the same defaults a tutorial would use, reaches 76.71% accuracy on `KDDTest+`, which is 23.16 points below the article's claim. A gap that size cannot be explained by differences in hyper-parameters. The decisive clue comes from cross-validation. Running 5-fold stratified CV on the training set gives a Random Forest F1 of 0.9988, almost exactly the article's 0.9987. In other words, the article's number is reproducible only when the model is scored on data it has already seen.

This is the classic mistake of reporting in-sample performance as if it were generalisation performance. It is especially damaging on NSL-KDD, because `KDDTest+` was deliberately built to contain attack types that are rare or absent in training. That makes it a genuine out-of-distribution test, the kind of novel traffic a deployed IDS actually faces. Skipping that test removes the only part of the benchmark that measures real-world capability, so the 99.87% figure says nothing useful about IDS performance.

2.2 SVM did not perform well

Across all seven metrics, Linear SVM was the worst of the six models (accuracy 74.97%, F1 0.7351, MCC 0.557, ROC-AUC 0.872). It sits just behind Logistic Regression and well behind the tree ensembles. Calling it a strong performer is not supported by anything I could measure. I used LinearSVC rather than a kernel SVM because kernel SVM on 125k samples is computationally impractical. A kernel version would be slower, but there is no reason to expect it to flip the ranking.

2.3 Naïve Bayes was not the worst model

Gaussian Naïve Bayes reached 77.16% accuracy and F1 0.7692, the second-best F1 in the study and clearly ahead of both Logistic Regression and SVM. It is one of the stronger threshold-based performers, not the weakest, which directly contradicts the article.

2.4 Metrics named but never reported

The article says it uses Precision, Recall and F1, yet gives no value for any of them, for any model. For an IDS this is the most important omission, because accuracy on its own cannot show how many attacks are missed. Without recall, a reader has no way to tell whether the model is operationally useful or dangerously blind.

2.5 The evaluation methodology

The methodology is not appropriate for the problem. On a near-balanced dataset dominated by one trivially separable attack (neptune, a SYN flood), accuracy is an easy and misleading headline. A sound evaluation would score on the held-out test set, report recall and a recall-weighted measure such as F2, include a threshold-independent measure like ROC-AUC and an imbalance-robust summary like MCC, and break errors down by attack category. The article does none of this.

2.6 Whether the conclusions hold

The conclusion that Random Forest is a near-perfect detector is not justified. The headline accuracy collapses under proper evaluation, the comparative ranking of SVM and Naïve Bayes is incorrect, and the metrics that matter most for security are missing. The article reads as a plausible tutorial, but its claims do not survive reproduction.

3. Exploratory Data Analysis (Reproduction)

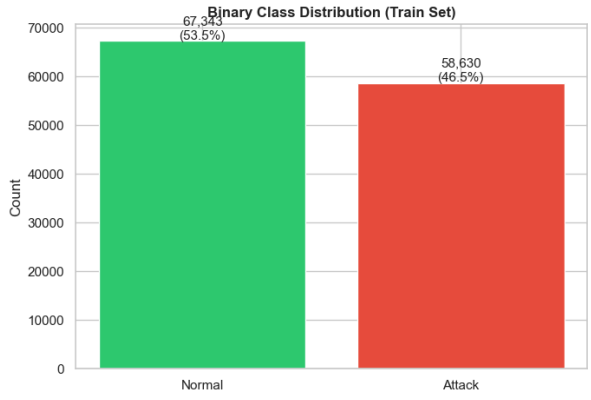
The article shows no EDA, so I performed my own. The findings here also motivate several of the modelling choices made later.

3.1 Data integrity

- The training set has 125,973 rows and the test set 22,544, each with 43 columns (24 integer, 15 float, 4 object).
- There are no missing values and no duplicate rows, which is consistent with how NSL-KDD was constructed.
- One feature, `num_outbound_cmds`, is constant (all zeros), so it carries no information and is removed.
- `difficulty_level` is curation metadata rather than a network property, so it is dropped to avoid leakage.

3.2 Class balance and why it is misleading

The training set is close to balanced: 67,343 normal connections (53.5%) against 58,630 attacks (46.5%). Real networks look nothing like this, since attacks there are usually well under 1% of traffic. Two things follow. First, accuracy is artificially easy to reach on this data, so a high accuracy number tells you little about deployment. Second, the dominant attack is neptune, with 41,214 records. A SYN flood like neptune is trivial to separate from normal traffic, and its sheer volume inflates any accuracy-based comparison. A model tuned to this balance will tend to over-alert once it meets genuinely imbalanced production traffic.



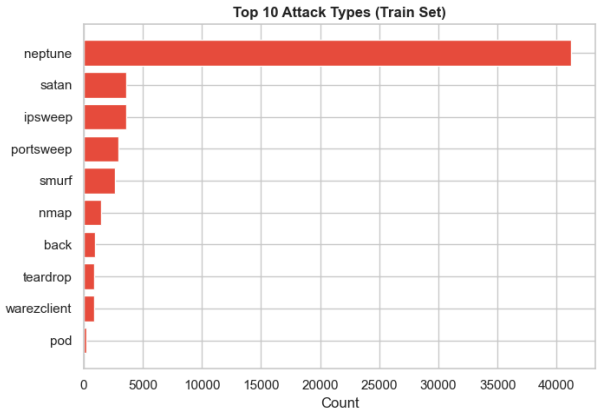


Figure 1. Binary class balance (left) and the ten most frequent attack types (right). neptune dominates.

3.3 Attack categories

Grouping the textual labels into the four standard families exposes a severe imbalance that the binary view hides:

Category	Train count	Note
Normal	67,343	baseline traffic
DoS	45,927	dominated by neptune
Probe	11,656	scanning / reconnaissance
R2L	995	remote-to-local; rare, dangerous
U2R	52	user-to-root; extremely rare

R2L and U2R together are well under 1% of the training data, yet they represent successful exploitation, exactly the events a security team most needs to catch. This imbalance is what later drives the error analysis in Section 7.

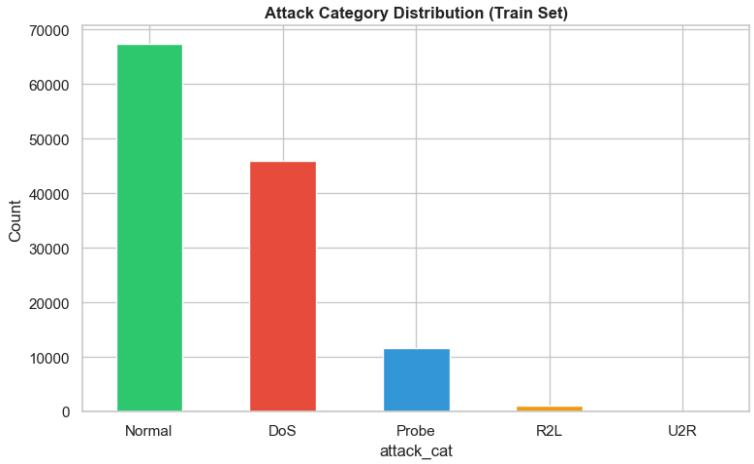


Figure 2. Attack-category distribution (training set). R2L and U2R are barely represented.

3.4 Feature distributions and skew

Most numerical features are heavily right-skewed. The large majority of connections sit near zero, while a small number, usually attacks, produce extreme byte counts or packet rates. Several features have a large share of IQR outliers, for example srv_diff_host_rate at 22.5%, dst_host_same_src_port_rate at 19.9%, and dst_bytes at 18.7%. These extreme values are not data errors. They are the signal, because attacks are precisely what generate unusual traffic. This pattern rules out Pearson correlation and most parametric assumptions.

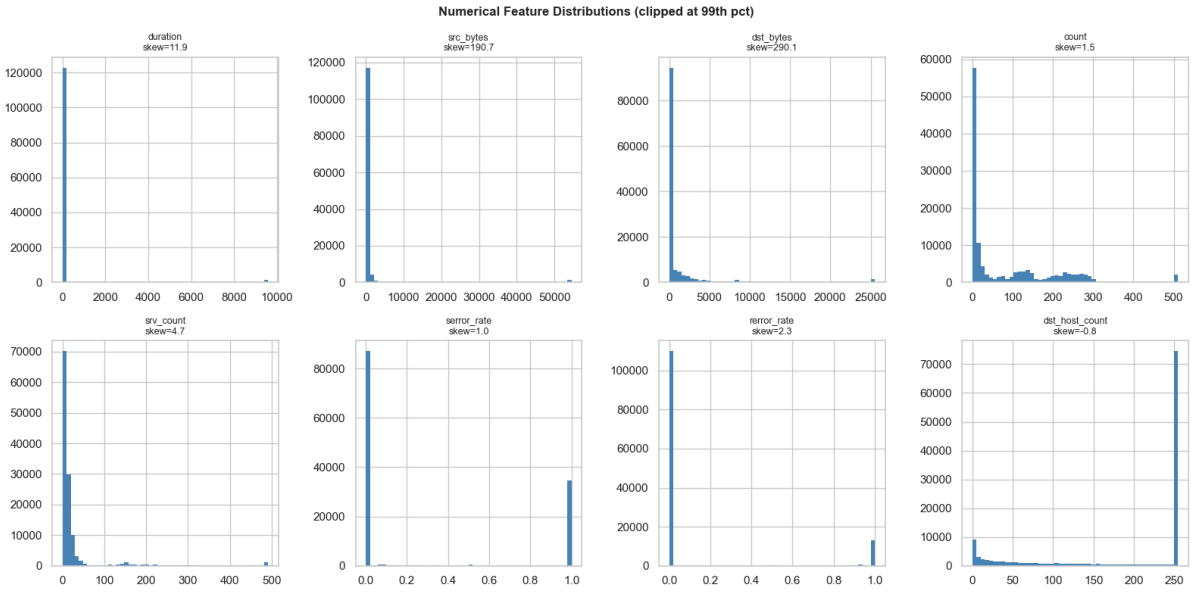


Figure 3. Distributions of eight representative numerical features (clipped at the 99th percentile). Skew values are large and positive.

3.5 Correlation analysis and the choice of Spearman

I considered three correlation coefficients. Pearson measures linear association but assumes roughly normal, outlier-free data, which is violated here. Kendall's tau is robust and rank-based, but it is expensive on 126k rows and is mainly useful for small samples. Spearman is rank-based, makes no normality assumption, tolerates the heavy outliers documented above, and captures any monotonic relationship, not just a linear one. Given skewed, non-normal, outlier-rich features, Spearman is the right tool, and it is what I use throughout.

Spearman flagged nine feature pairs with |r| at or above 0.90, for example serror_rate against srv_serror_rate (r = 0.973) and rerror_rate against srv_rerror_rate (r = 0.966). These near-duplicates are the redundancy I deal with in Section 4. The correlations are not statistical accidents either. SYN-error rates measured per connection and per service rise together because a single SYN flood drives both at once, which is meaningful in security terms.

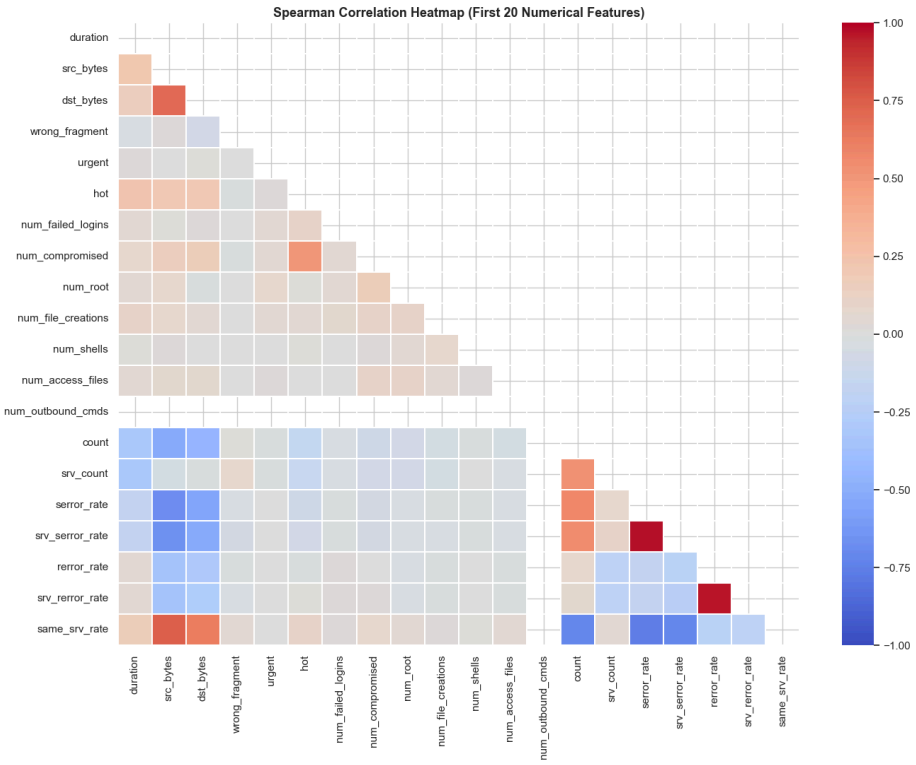


Figure 4. Spearman correlation heatmap (first 20 numerical features). Bright blocks mark redundant feature clusters.

3.6 Crosstab analysis

Grouping by protocol and flag confirms that even simple structure is highly informative. By protocol, ICMP traffic is dominated by Probe (49.9%) and DoS (34.3%), whereas UDP is 82.9% normal. By TCP connection flag the separation is sharper still. Flags such as RSTOS0 (100% attack), SH (99.3%) and S0 (99.0%) are almost perfect indicators of an attack, while SF, the normal-completion flag, is only 15.4% attack. This is why even simple models reach moderate accuracy, and why flag turns out to be a top feature for the tree models.

4. Feature Engineering Analysis

4.1 Feature engineering in the source

The article describes no feature engineering. It gives no encoding scheme for the three categorical features, no scaling, no feature selection, and no treatment of the constant feature. Tree models can absorb some of these omissions, but scale-sensitive models such as Logistic Regression, SVM and Gaussian Naïve Bayes cannot, and leaving in redundant or zero-variance features distorts the feature-importance numbers. This is a real gap in the original work.

4.2 What I did, and why

1. Binary target. Every non-normal label maps to 1 (attack) and normal maps to 0, matching the article's framing.

2. Dropping non-informative columns. I removed the constant num_outbound_cmds, which has zero variance and only adds noise for the linear and distance models, and the metadata column difficulty_level, which is not a network property and risks leakage.
3. Label encoding for categoricals. protocol_type has 3 values, flag has 11, and service has about 70. One-hot encoding service alone would add roughly 69 sparse columns. Tree models read the encoded integers as arbitrary split points with no ordinal meaning, and for the linear and distance models the later standardisation softens any spurious ordering. Test-set services that never appear in training are mapped to −1.
4. Redundant feature removal. Using |Spearman r| > 0.95, I dropped srv_serror_rate and srv_rerror_rate, taking the feature count from 40 to 38. At that threshold two features are almost perfectly monotonically related, so removing one loses essentially no signal while cutting dimensionality and sharpening the importance estimates.
5. Scaling only where it helps. StandardScaler, fit on the training data only, is applied for Logistic Regression, Linear SVM and Gaussian NB, which are scale-sensitive. The tree models (Decision Tree, Random Forest, XGBoost) are scale-invariant and use the unscaled features.

4.3 Spotting and handling redundancy

I detected redundancy by computing the absolute Spearman correlation matrix, taking its upper triangle, and selecting any feature whose correlation with an earlier feature exceeded 0.95. Two features with |r| near 1 are monotonically collinear: they span almost the same direction of variation, so the second one adds little independent information while diluting the importance attributed to the pair. The fix is to keep one representative from each redundant cluster. The same idea extends to variance-inflation analysis or to a PCA step, although for an IDS that has to be explainable I prefer dropping named features over rotating into latent components.

4.4 Was the feature engineering meaningful?

Yes, on both the mathematical side and the security side. Mathematically, removing zero-variance and near-duplicate features lowers noise and improves the conditioning of the linear models, and standardisation lets the gradient-based and distance-based methods converge and weigh features on a common scale. On the security side, the features that remain are still directly interpretable. serror_rate spikes during SYN floods, logged_in separates authenticated sessions from probes, and dst_host_same_src_port_rate flags port scans. That interpretability matters when an analyst has to justify why a given connection was blocked.

4.5 Additional features that could help

- Ratio features such as src_bytes / (dst_bytes + 1), which capture the asymmetric flows typical of exfiltration or scanning.
- Log transforms (log1p) of the heavily skewed byte and count features, to linearise their effect for the parametric models.

- Frequency or target encoding of service in place of label encoding, giving the linear models a more meaningful ordering.
- Aggregated session features such as the entropy of destination ports or distinct-service counts, to strengthen detection of the low-volume R2L and U2R attacks that are currently missed most often.

5. Reproducibility Analysis

5.1 Executing the original code

There is no original code to execute. The article contains no code, no notebook, and no public repository, so nothing can be re-run directly. Reproduction was only possible by re-implementing the whole pipeline from the textual description and from standard knowledge of NSL-KDD.

5.2 Files and dependencies

The dataset itself is public (NSL-KDD, mirrored on GitHub), so the inputs can be obtained. The software environment is not stated anywhere in the article. My own environment is fully pinned so that my results can be reproduced: Python with `numpy 1.26.4, pandas 2.2.3, scikit-learn 1.5.2, xgboost 3.2.0`, a fixed random seed of 42, and the official KDDTrain+ / KDDTest+ split.

5.3 Hidden preprocessing

There are hidden steps, unavoidably, because none are written down. Categorical encoding, scaling, handling of the constant feature, the train/test split, and the validation scheme are all left implicit. Each of these choices changes the results, and the most consequential one is the evaluation protocol itself. The evidence indicates the reported number is a training-set score rather than a held-out test score.

5.4 Overall reproducibility

Reproducibility is low. The dataset can be reproduced, but the result cannot. Without code, a stated split, or preprocessing details, the 99.87% claim cannot be verified independently, and when the pipeline is rebuilt with defensible choices the claim does not hold. This is the central reproducibility failure of the source.

6. Experimental Results

6.1 Experimental design and modifications

I trained the five models named by the article (Logistic Regression, Decision Tree, Random Forest, SVM and Naïve Bayes) and added XGBoost as a strong gradient-boosting baseline. The Linear SVM is wrapped in probability calibration so that a ROC-AUC can be computed for it. Every model is trained on KDDTrain+ and evaluated once on the untouched KDDTest+, all with a fixed seed. Beyond accuracy I report seven metrics, and I run 5-fold stratified cross-validation on the training set to expose the train/test gap.

6.2 Test-set results

Model	Acc.	Prec.	Rec.	F1	F2	MCC	AUC
XGBoost	81.00%	0.969	0.688	0.805	0.731	0.665	0.961
Naïve Bayes	77.16%	0.906	0.668	0.769	0.705	0.578	0.826
Decision Tree	77.63%	0.966	0.629	0.762	0.677	0.615	0.801
Random Forest	76.71%	0.966	0.612	0.750	0.661	0.602	0.963
Logistic Regression	75.08%	0.924	0.613	0.737	0.657	0.558	0.873
SVM (Linear)	74.97%	0.925	0.610	0.735	0.655	0.557	0.872

XGBoost wins on every threshold metric. One detail in the table is worth pausing on. Random Forest has very high precision (0.966) but low recall (0.612), so at the default threshold it misses many attacks, even though its ROC-AUC of 0.963 is the highest in the study. That combination means Random Forest ranks attacks well but its 0.5 decision threshold is badly placed for recall. A single accuracy number cannot reveal that.

6.3 The train/test gap (cross-validation)

5-fold stratified cross-validation on the training set produces the F1 scores below. The contrast with the test-set F1 column is the quantitative heart of the reproduction failure.

Model	Train CV F1 (mean)	Test F1	Gap
Random Forest	0.9988	0.750	0.249
XGBoost	0.9985	0.805	0.194
Decision Tree	0.9977	0.762	0.236
SVM (Linear)	0.9512	0.735	0.216
Logistic Regression	0.9486	0.737	0.212
Naïve Bayes	0.8845	0.769	0.116

Random Forest's training CV F1 of 0.9988 reproduces the article's 0.9987 almost exactly, while its test F1 is 0.750. This is the strongest single piece of evidence that the published 99.87% is an in-sample number.

6.4 Confusion matrices

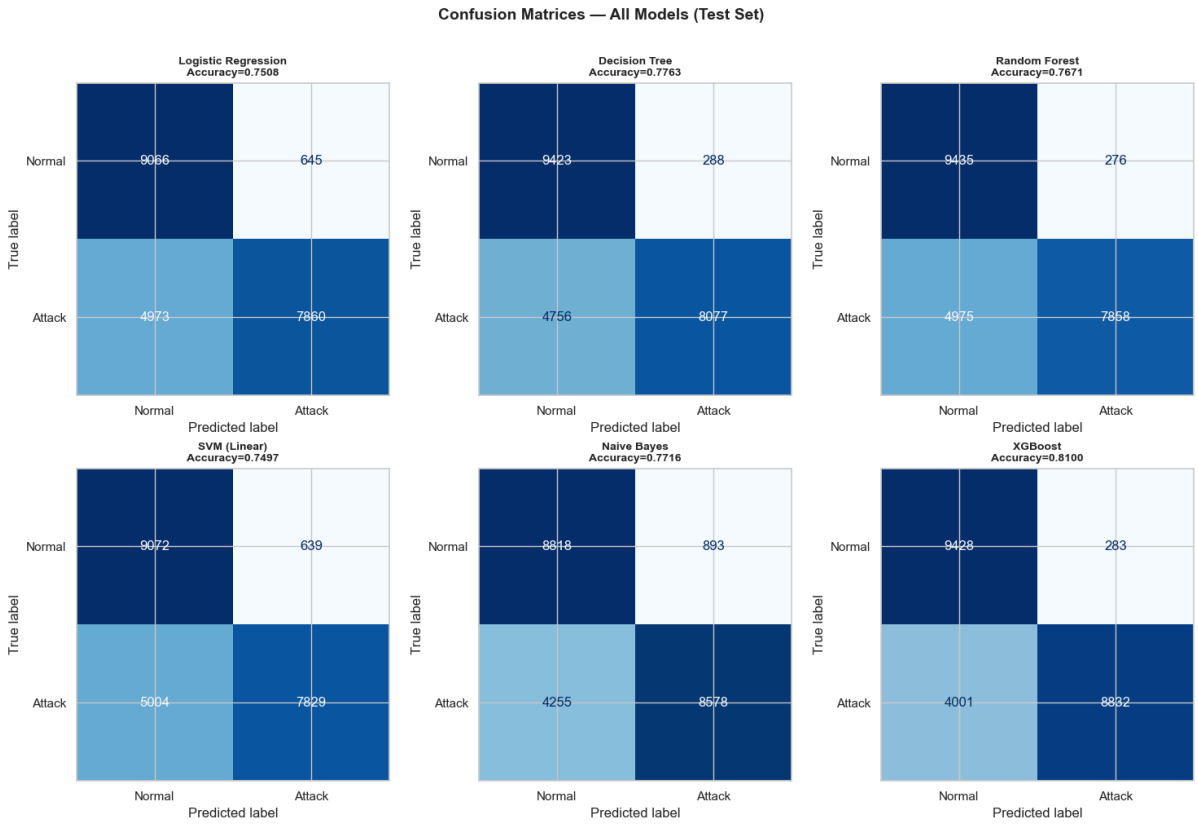


Figure 5. Confusion matrices for all six models on the test set. The large bottom-left cells are missed attacks (false negatives).

6.5 ROC curves

The ROC curves split the models into two tiers. The tree ensembles, Random Forest and XGBoost, sit near AUC 0.96, while the linear models and Naïve Bayes fall lower, in the 0.80 to 0.87 range. This reinforces the earlier point: Random Forest's weakness on the threshold metrics is a threshold-placement problem, not a ranking problem.

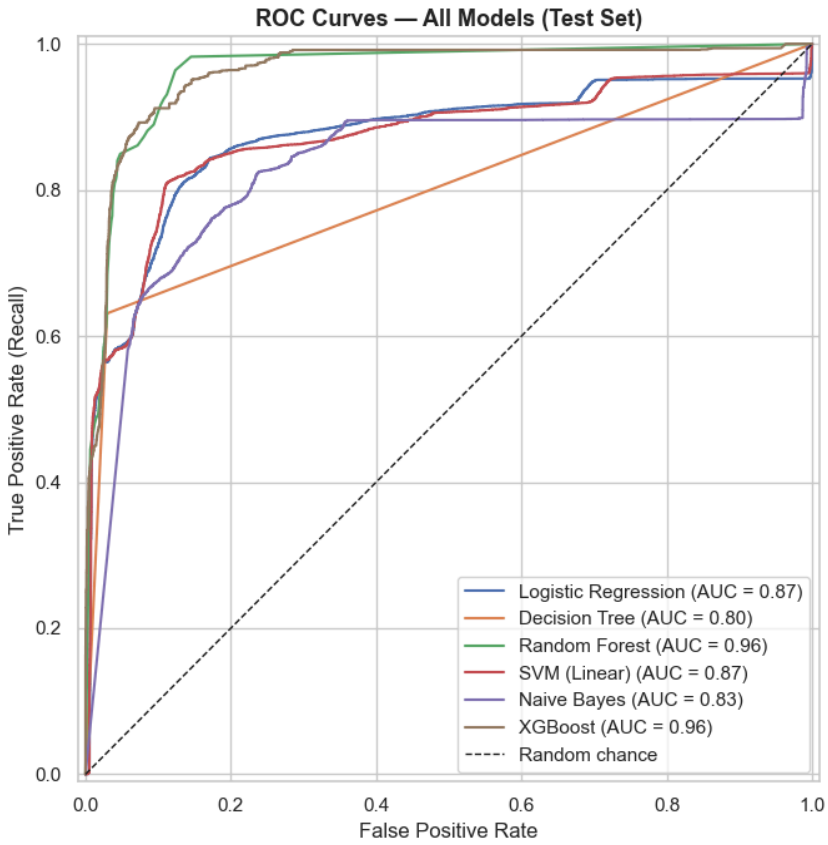


Figure 6. ROC curves (test set). Tree ensembles rank attacks substantially better than the linear models.

6.6 Feature importance

The Random Forest importances line up with the EDA. Connection flag, the same- and diff-service rate features, byte counts, and the SYN-error rates carry the most weight, which are exactly the quantities that distinguish floods, scans and normal completions. The dropped constant feature contributes nothing, which confirms the feature-engineering decision.

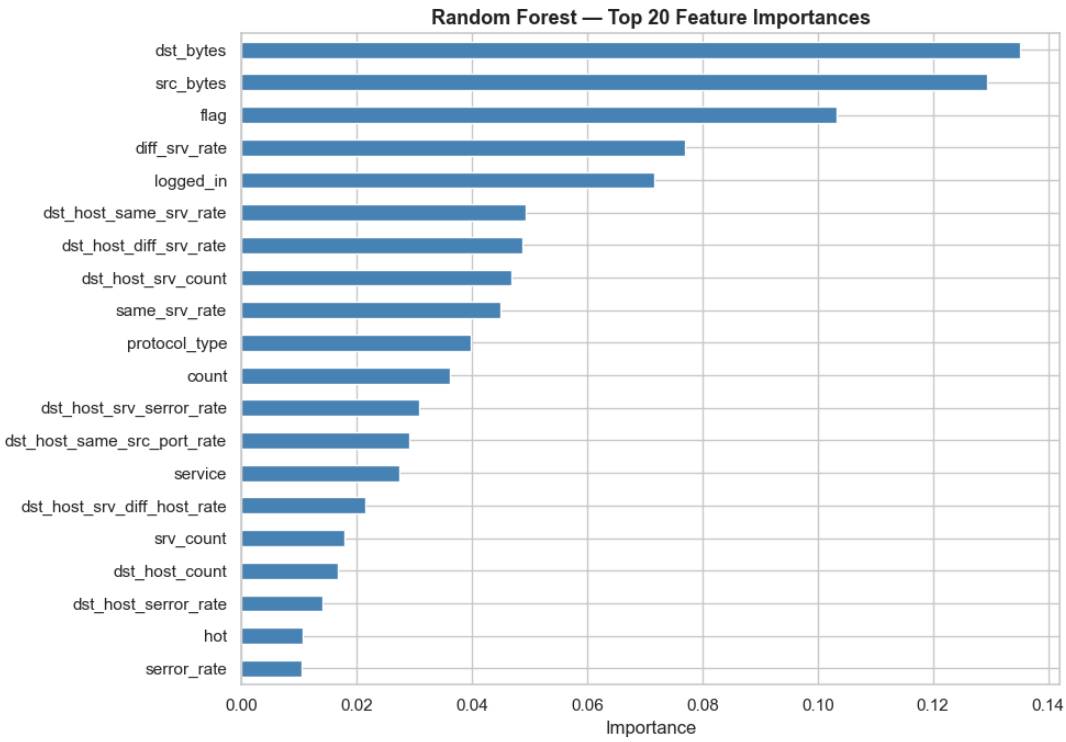


Figure 7. Random Forest top-20 feature importances. Flag, service-rate and error-rate features dominate.

7. Evaluation and Error Analysis

7.1 Why these metrics for an IDS

Metric	What it measures	Why it matters for an IDS
Accuracy	(TP+TN)/N	Intuitive, but misleading under imbalance and easy attacks
Precision	TP/(TP+FP)	Low precision means false alarms and analyst alert fatigue
Recall	TP/(TP+FN)	Low recall means missed attacks, the most dangerous failure
F1	harmonic mean of P, R	A balanced single summary
F2 (β=2)	weights recall twice	Matches the IDS preference for catching attacks
MCC	correlation of pred vs truth	Best single score under imbalance; ranges −1 to +1
ROC-AUC	area under the TPR–FPR curve	Threshold-independent ranking quality

Accuracy on its own, the article's only metric, is the weakest choice here. I keep it for comparability but lead with recall, F2 and MCC, which speak directly to operational risk.

7.2 Error breakdown (Random Forest)

On the test set, Random Forest produces 276 false positives (1.22%) and 4,975 false negatives (22.07%). False positives are low, so alert fatigue is not the main worry here. The dominant problem is missed attacks, and those misses are not spread evenly. They cluster in the rarest and most dangerous categories:

Category of missed attack	Count missed	Security meaning
R2L (remote-to-local)	2,621	remote login / credential compromise
DoS	1,671	service disruption
Probe	507	reconnaissance / scanning
U2R (user-to-root)	176	privilege escalation to root

The individual attacks missed most often include guess_passwd (1,231), warezmaster (818) and apache2 (710). R2L and U2R are exactly the categories that were under-represented in training (995 and 52 records), so the model has very little to learn from, and these happen to be the intrusions a defender least wants to miss.

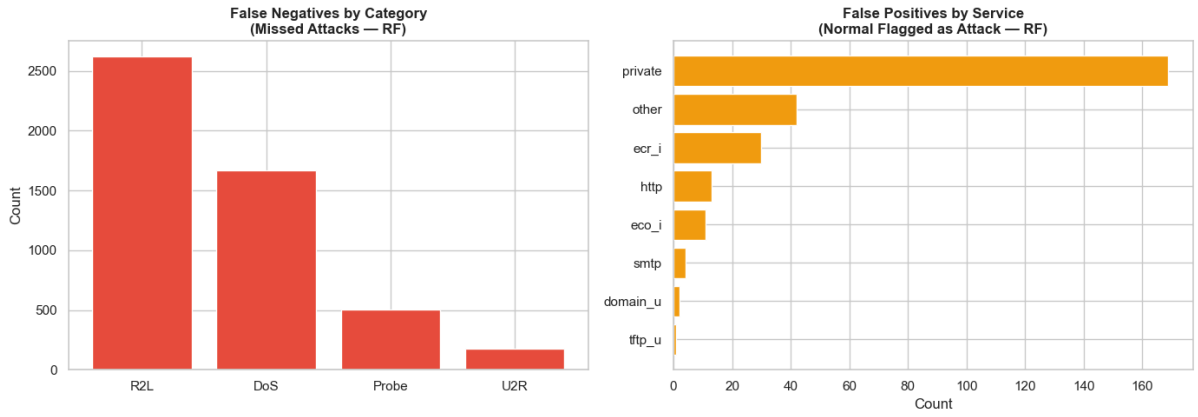


Figure 8. Missed attacks by category (left) and false positives by service (right) for Random Forest.

7.3 The false-positive / false-negative trade-off

In intrusion detection a false negative is usually far more costly than a false positive. An attack labelled normal gives the adversary time to escalate and move through the network, while a false alarm mostly costs analyst time. The reasonable response is to accept somewhat lower precision in exchange for higher recall, by lowering the decision threshold, adjusting class weights, or resampling, and to track F2 rather than F1 because F2 weights recall twice. The current 22% miss rate is too high for production and would be the first thing I would try to improve. The source article never discusses this trade-off, which is a serious omission for a security application.

8. Conclusions

8.1 Key findings

- The article's 99.87% Random Forest accuracy does not reproduce on the held-out test set, where it reaches 76.71%. It instead matches my training CV score, which points to an in-sample evaluation error.
- The comparative claims are wrong. SVM was the weakest model, and Naïve Bayes was one of the stronger ones, the opposite of what the article states.
- A realistic best result on NSL-KDD is XGBoost at 81.00% accuracy and F1 0.805, not around 99%.
- About 22% of attacks are missed, concentrated in R2L and U2R, and an accuracy-only report hides this completely.

8.2 Lessons learned

A few lessons stand out. Always evaluate on a held-out set. On imbalanced or skewed security data, accuracy is a trap, and recall, F2 and MCC are the metrics that matter. Reproducibility needs code, a stated split, and pinned dependencies. And breaking errors down by attack category turns an abstract score into a statement about real operational risk.

8.3 Strengths and weaknesses of the original work

The article has a few genuine strengths. It picks a relevant problem, uses a standard and well-understood dataset, and surveys a sensible set of baseline models. The weaknesses are what decide the matter. There is no code or repository, no stated split or preprocessing, an apparent in-sample evaluation, accuracy as the only metric, two incorrect comparative claims, and no treatment of imbalance or of the false-negative risk that defines the domain.

8.4 Suggestions for future improvement

- Publish the code, the exact split, and a pinned environment so the results can be checked.
- Report recall, F2, MCC and ROC-AUC, and tune the decision threshold or class weights to raise recall on R2L and U2R.
- Address imbalance directly, with class weights, focal loss, or resampling, and validate on a more modern dataset such as CIC-IDS2017 before making any claim about production readiness.
- Add the engineered features from Section 4.5 to strengthen detection of the low-volume attacks.

8.5 Should this work be used as a reference?

No. The claims are not backed by reproducible evidence, and the evaluation methodology is unsound for a security task. The article is a useful illustration of a common pitfall, reporting training accuracy as if it were generalisation, but it should not be used as a methodological reference for building or evaluating an intrusion detection system. The pipeline I built here is reproducible, produces realistic and operationally honest results, and could serve as a baseline for similar binary-IDS problems.